

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 «Компьютерные науки и прикладная математика»
Кафедра Вычислительной математики и программирования

Лабораторная работа:
по дисциплине
“Численные методы”

Студентка:

Хакимова Александра Эльдаровна

Группа: М8О-305Б-22

Преподаватель:

Киндинова Виктория Валерьевна

Дата:

Москва, 2025

Лабораторная работа №1

Вариант 20

1.1. Реализовать алгоритм LU - разложения матриц (с выбором главного элемента) в виде программы. Используя разработанное программное обеспечение, решить систему линейных алгебраических уравнений (СЛАУ). Для матрицы СЛАУ вычислить определитель и обратную матрицу.

$$\begin{cases} 7 \cdot x_1 + 8 \cdot x_2 + 4 \cdot x_3 - 6 \cdot x_4 = -126 \\ -x_1 + 6 \cdot x_2 - 2 \cdot x_3 - 6 \cdot x_4 = -42 \\ 2 \cdot x_1 + 9 \cdot x_2 + 6 \cdot x_3 - 4 \cdot x_4 = -115 \\ 5 \cdot x_1 + 9 \cdot x_2 + x_3 + x_4 = -67 \end{cases}$$

Код:

```
import numpy as np

def scale_matrix(A):
    """
    Функция для нормализации строк матрицы A по диагональным элементам.
    Возвращает матрицу, в которой каждая строка нормализована по её диагональному
    элементу.
    """
    norm_mat = np.copy(A)

    # Проходим по каждой строке матрицы.
    for i in range(norm_mat.shape[0]):
        diag_element = norm_mat[i, i] # Берем диагональный элемент текущей строки.

        # Проходим по каждому элементу и нормализуем его.
        for j in range(norm_mat.shape[1]):
```

```
        norm_mat[i, j] /= diag_element # Делим каждый элемент на диагональный элемент.
```

```
    return norm_mat # Возвращаем нормализованную матрицу.
```

```
def calculate_inverse1(A):
```

```
    """
```

```
    Функция для вычисления обратной матрицы с прямым методом Гаусса.
```

```
    Возвращает матрицу, которая является обратной к матрице A.
```

```
    """
```

```
    A_copy = np.copy(A)
```

```
    identity_matrix = np.eye(A_copy.shape[0]) # Создаем единичную матрицу того же размера, что и A.
```

```
    # Проходим по каждой строке матрицы.
```

```
    for i in range(A_copy.shape[0]):
```

```
        # Проходим по элементам строки, исключая диагональный элемент.
```

```
        for j in range(A_copy.shape[1]):
```

```
            if i != j:
```

```
                identity_matrix[j, :] -= (A[j, i] * identity_matrix[i, :]) #
```

```
Обновляем строки единичной матрицы.
```

```
    return identity_matrix # Возвращаем полученную матрицу.
```

```
def calculate_inverse2(A):
```

```

"""

Функция для вычисления обратной матрицы с обратным методом Гаусса.

Возвращает матрицу, которая является обратной к матрице A.

"""

A_copy = np.copy(A)  # Копируем матрицу A.

identity_matrix = np.eye(A_copy.shape[0])  # Создаем единичную матрицу того же
размера, что и A.

# Проходим по строкам в обратном порядке.

for i in range(A_copy.shape[0] - 1, -1, -1):

    # Проходим по элементам строки в обратном порядке, исключая диагональный
элемент.

    for j in range(A_copy.shape[1] - 1, -1, -1):

        if i != j:

            identity_matrix[j, :] -= (A[j, i] * identity_matrix[i, :])  #
Обновляем строки единичной матрицы.

return identity_matrix  # Возвращаем полученную матрицу.

```

```

def matrix_multiplication(A, B):

```

```

"""

Функция для умножения матриц A и B.

Возвращает результат произведения двух матриц.

"""

result_matrix = np.zeros((A.shape[0], B.shape[1]))  # Создаем матрицу для
хранения результата умножения.

```

```

# Проходим по строкам первой матрицы A.

for i in range(result_matrix.shape[0]):

    # Проходим по столбцам второй матрицы B.

    for j in range(result_matrix.shape[1]):

        # Перемножаем строки A на столбцы B.

        for k in range(A.shape[1]):

            result_matrix[i, j] += A[i, k] * B[k, j] # Суммируем произведения
для получения элемента в результирующей матрице.


return result_matrix # Возвращаем результирующую матрицу.


def LU_decomposition(A):

    """

    Функция для разложения матрицы A на множители L и U.

    Возвращает нижнюю (L) и верхнюю (U) треугольные матрицы.

    """

    A_copy = np.copy(A) # Копируем матрицу A.

    L = np.eye(A.shape[0]) # Создаем начальную единичную матрицу для L.

    U = np.copy(A) # U изначально копирует A.


    # Проходим по строкам и столбцам для нахождения множителей.

    for i in range(A_copy.shape[0]):

        for j in range(i + 1, A_copy.shape[0]):

```

```

        factor = A_copy[j, i] / A_copy[i, i] # Рассчитываем множитель для
текущей строки.

        L[j, i] = factor # Сохраняем множитель в матрице L.

# Обновляем строки матрицы U.

for k in range(A.shape[1]):

    A_copy[j, k] -= factor * A_copy[i, k] # Вычитаем произведение из
строк matrix.

    U[j, k] = A_copy[j, k] # Обновляем значения для U.

return L, U # Возвращаем матрицы L и U.

def solve_SLAU(L, U, B):

    """

    Функция для решения системы линейных алгебраических уравнений (СЛАУ) в виде
LU-разложения.

    Возвращает вектор решения x.

    """

    y = np.zeros(len(B)) # Инициализируем вспомогательный вектор y.

    x = np.zeros(len(B)) # Инициализируем решение x.

    # Прямой ход для нахождения y.

    for i in range(len(B)):

        sum_L = 0.0 # Инициализируем сумму.

        for j in range(i):

```

```

        sum_L += L[i][j] * y[j] # Суммируем произведения.

    y[i] = B[i] - sum_L # Находим текущий элемент y.

# Обратный ход для нахождения x.

for i in range(len(B) - 1, -1, -1):

    temp = y[i] # Сохраняем текущий элемент y.

    for j in range(i + 1, len(B)):

        temp -= U[i][j] * x[j] # Суммируем произведения.

    x[i] = temp / U[i][i] # Нормализуем текущий элемент x.

return x # Возвращаем найденный вектор x.

def compute_determinant(L, U):

    """

    Функция для вычисления детерминанта матрицы A через матрицы L и U.

    Возвращает значение детерминанта.

    """

    detL = np.prod(np.diag(L)) # Вычисляем детерминант L как произведение
диагональных элементов L.

    detU = np.prod(np.diag(U)) # Вычисляем детерминант U как произведение
диагональных элементов U.

    return detL * detU # Детерминант матрицы A равен произведению детерминантов L и
U.

```

```

def calculate_inverse_A(L, U):

    """

    Функция для вычисления обратной матрицы A с использованием матриц L и U.

    Возвращает обратную матрицу.

    """

    L_inverse = np.copy(calculate_inverse1(L)) # Вычисляем обратную матрицу L.

    U_normalized = scale_matrix(U) # Нормализуем матрицу U.

    U_inverse = np.copy(calculate_inverse2(U_normalized)) # Вычисляем обратную
матрицу U.

    # Умножаем обратные матрицы U и L для получения обратной матрицы A.

    inverted_matrix = matrix_multiplication(U_inverse, L_inverse)

    return inverted_matrix # Возвращаем обратную матрицу A.

def main(A):

    """

    Основная функция для обработки матрицы A и решения систем уравнений.

    Выполняет разложение, решает СЛАУ и вычисляет обратную матрицу и детерминант.

    """

    coefficient_matrix = A[:, :-1] # Извлекаем коэффициенты (все столбцы, кроме
последнего).

    result_vector = A[:, -1] # Извлекаем результаты (последний столбец).

    # Выполняем LU-разложение на коэффициентной матрице.

```

```

L, U = LU_decomposition(coefficient_matrix)

print("Матрица L:\n", L) # Выводим матрицу L.

print("Матрица U:\n", U) # Выводим матрицу U.

# Решаем систему уравнений и выводим результат.

print("Решение СЛАУ:\n", solve_SLAU(L, U, result_vector))

# Вычисляем детерминант матрицы A и выводим его.

print("Детерминант матрицы A:\n", compute_determinant(L, U))

# Вычисляем и выводим обратную матрицу A.

print("Обратная матрица A:\n", calculate_inverse_A(L, U))

# Исходная матрица для тестирования.
A = np.array([[7, 8, 4, -6, -126],
              [-1, 6, -2, -6, -42],
              [2, 9, 6, -4, -115],
              [5, 9, 1, 1, -67]])

# Запускаем основную функцию с заданной матрицей A.

main(A)

```

Вывод:

Матрица L:

$$\begin{bmatrix} 1. & 0. & 0. & 0. & \\ -0.14285714 & 1. & 0. & 0. & \\ 0.28571429 & 0.85714286 & 1. & 0. & \\ 0.71428571 & 0.42857143 & 0. & 1. & \end{bmatrix}$$

Матрица U:

$$\begin{bmatrix} 7 & 8 & 4 & -6 \\ 0 & 7 & -1 & -6 \\ 0 & 0 & 4 & 3 \\ 0 & 0 & 0 & 7 \end{bmatrix}$$

Решение СЛАУ:

$$[-0.1574344 \quad -4.33673469 \quad -12.1122449 \quad 6.95918367]$$

Детерминант матрицы A:

$$1372.0$$

Обратная матрица A:

$$\begin{bmatrix} 0.85714286 & -1. & 0. & 0. & \\ 0.14285714 & 1. & 0. & 0. & \\ -0.40816327 & -0.85714286 & 1. & 0. & \\ -0.7755102 & -0.42857143 & 0. & 1. & \end{bmatrix}$$

1.2. Реализовать метод прогонки в виде программы, задавая в качестве входных данных ненулевые элементы матрицы системы и вектор правых частей. Используя разработанное программное обеспечение, решить СЛАУ с трехдиагональной матрицей.

$$\begin{cases} -6 \cdot x_1 + 6 \cdot x_2 = 30 \\ 2 \cdot x_1 + 10 \cdot x_2 - 7 \cdot x_3 = -31 \\ -8 \cdot x_2 + 18 \cdot x_3 + 9 \cdot x_4 = 108 \\ 6 \cdot x_3 - 17 \cdot x_4 - 6 \cdot x_5 = -114 \\ 9 \cdot x_4 + 14 \cdot x_5 = 124 \end{cases}$$

Код:

```
import numpy as np

def construct_matrix(X, Y):
    """
    Эта функция формирует квадратную матрицу на основе заданных векторов X и Y.
    Каждая строка матрицы содержит элементы, основанные на элементе X и его соседях.

    Параметры:
    X -- Одинарный массив, содержащий значения, которые используются для заполнения
    матрицы.
    Y -- Одинарный массив, который, по сути, задает размеры выходной матрицы.

    Возвращает:
    matrix -- Созданная квадратная матрица размером len(Y) x len(Y).
    """
    # Создаем матрицу, заполненную нулями, размером len(Y) на len(Y).
    matrix = np.zeros((len(Y), len(Y)))

    for idx in range(matrix.shape[0]):
        if idx == 0:
            # Заполняем первую строку матрицы первыми двумя элементами из X.
            matrix[idx, 0] = X[0]
            matrix[idx, 1] = X[1]
            pos = 2 # Устанавливаем начальную позицию для доступа к X.
        elif idx == len(matrix) - 1:
            # Заполняем последнюю строку матрицы последними двумя элементами из X.
            matrix[idx, len(matrix) - 1] = X[-1]
            matrix[idx, len(matrix) - 2] = X[-2]
        else:
            # Заполняем элементы матрицы на основе текущего индекса и его соседей в
            X.

            matrix[idx, idx] = X[pos + 1]
            matrix[idx, idx - 1] = X[pos]
            matrix[idx, idx + 1] = X[pos + 2]
            pos += 3 # Увеличиваем позицию для доступа к X.

    return matrix # Возвращаем сформированную матрицу.

def solve_system(matrix, results):
    """
    Эта функция решает систему линейных уравнений, заданную матрицей coefficients
    и вектором результатов.

    Параметры:
    matrix -- Квадратная матрица коэффициентов системы.
```

```

results -- Вектор результатов, соответствующий системе уравнений.

Возвращает:
solutions -- Вектор решений, полученных для системы уравнений.
"""

coeffs = np.zeros(matrix.shape[0]) # Массив для хранения коэффициентов.
constants = np.zeros(matrix.shape[0]) # Массив для хранения констант.
solutions = np.zeros(matrix.shape[0]) # Массив для хранения решений.

# Начальные условия для первого уравнения.
k = matrix[0, 0] # Первый элемент главной диагонали.
coeffs[0] = -matrix[0, 1] / k # Вычисляем первый коэффициент. (-c1/b1)
constants[0] = results[0] / k # Вычисляем первую (d1/b1)

for idx in range(len(matrix) - 1):
    # Для всех уравнений, кроме последнего, проводим итерации.
    k = matrix[idx, idx] + matrix[idx, idx - 1] * coeffs[idx - 1] # Обновляем
k.
    coeffs[idx] = -matrix[idx, idx + 1] / k # Вычисляем коэффициенты.
    constants[idx] = (results[idx] - matrix[idx, idx - 1] * constants[idx - 1])
/ k # Константы.

# Обработка последнего уравнения отдельно.
last_val = results[len(matrix) - 1] - matrix[len(matrix) - 1, len(matrix) - 2] *
constants[len(matrix) - 2]
last_coeff = matrix[len(matrix) - 1, len(matrix) - 1] + matrix[len(matrix) - 1,
len(matrix) - 2] * coeffs[len(matrix) - 2]
solutions[len(matrix) - 1] = last_val / last_coeff # Решаем последнее
уравнение.

# Обратный проход для вычисления всех решений.
for idx in range(len(matrix) - 2, -1, -1):
    solutions[idx] = coeffs[idx] * solutions[idx + 1] + constants[idx] #
Вычисляем решения.

return solutions # Возвращаем полученные решения.

def execute(A, B):
    """
    Основная функция для выполнения заполнения матрицы и решения системы уравнений.

    Параметры:
    A -- Одинарный массив, содержащий элементы для формирования матрицы.
    B -- Одинарный массив, содержащий результаты, соответствующие системе уравнений.
    """

```

```

matrix = construct_matrix(A, B) # Создаем матрицу.
print("Сформированная матрица:")
print(matrix) # Выводим построенную матрицу.

solutions = solve_system(matrix, B) # Решаем систему уравнений.
print("Решения:")
print(solutions) # Выводим решения.

# Примеры входных данных для тестирования функций.
A = np.array([-6.0, 6.0, 2.0, 10.0, -7.0, -8.0, 18.0, 9.0, 6.0, -17.0, -6.0, 9.0, 14.0])
B = np.array([30.0, -31.0, 108.0, -114.0, 124.0])

# Запускаем основную функцию с заданными массивами A и B.
execute(A, B)

```

Вывод:

Сформированная матрица:

```

[[ -6.  6.  0.  0.  0.]
 [  2. 10. -7.  0.  0.]
 [  0. -8. 18.  9.  0.]
 [  0.  0.  6. -17. -6.]
 [  0.  0.  0.  9. 14.]]

```

Решения:

```

[-5.  0.  3.  6.  5.]

```

1.3. Реализовать метод простых итераций и метод Зейделя в виде программ, задавая в качестве входных данных матрицу системы, вектор правых частей и точность вычислений. Используя разработанное программное обеспечение, решить СЛАУ. Проанализировать количество итераций, необходимое для достижения заданной точности.

$$\begin{cases} 10 \cdot x_1 - x_2 - 2 \cdot x_3 + 5 \cdot x_4 = -99 \\ 4 \cdot x_1 + 28 \cdot x_2 + 7 \cdot x_3 + 9 \cdot x_4 = 0 \\ 6 \cdot x_1 + 5 \cdot x_2 - 23 \cdot x_3 + 4 \cdot x_4 = 67 \\ x_1 + 4 \cdot x_2 + 5 \cdot x_3 - 15 \cdot x_4 = 58 \end{cases}$$

Код:

```

import numpy as np # Импорт библиотеки для работы с матрицами и числовыми операциями

def normalize_matrix(mat):
    """
    Нормализует матрицу для итерационных методов решения СЛАУ.
    Разделяет коэффициенты на диагональные элементы и изменяет знаки.
    """

```

```

"""
normalized = np.copy(mat) # Создаем копию исходной матрицы
for idx in range(normalized.shape[0]): # Проходим по каждой строке
    pivot = normalized[idx, idx] # Берем диагональный элемент
    normalized[idx, idx] = 0 # Обнуляем диагональный элемент
    for j in range(normalized.shape[1]): # Проходим по каждому столбцу
        if idx != j and j != normalized.shape[1] - 1:
            # Нормируем недиагональные элементы (кроме последнего столбца)
            normalized[idx, j] = normalized[idx, j] / -pivot
        elif j == normalized.shape[1] - 1:
            # Нормируем элемент последнего столбца (свободные члены)
            normalized[idx, j] = normalized[idx, j] / pivot
    return normalized

def find_max_row_sum(mat):
    """
    Находит максимальную сумму абсолютных значений элементов в строках матрицы.
    Используется для оценки нормы матрицы.
    """
    max_sum = 0
    for row in range(mat.shape[0]): # Проходим по каждой строке
        row_sum = np.sum(np.abs(mat[row, :])) # Сумма абсолютных значений строки
        if row_sum > max_sum:
            max_sum = row_sum # Обновляем максимум
    return max_sum

def compute_inverse(mat):
    """
    Вычисляет обратную матрицу методом элементарных преобразований.
    """
    identity = np.eye(mat.shape[0]) # Создаем единичную матрицу
    for i in range(mat.shape[0]): # Прямой ход метода Гаусса
        for j in range(mat.shape[1]):
            if i != j:
                # Вычитаем из других строк текущую строку с коэффициентом
                identity[j, :] -= mat[j, i] * identity[i, :]
    return identity

def matrix_product(mat1, mat2):
    """
    Умножение двух матриц (реализация вручную).
    """
    result = np.zeros((mat1.shape[0], mat2.shape[1])) # Инициализация результата
    for row in range(result.shape[0]): # Проход по строкам
        for col in range(result.shape[1]): # Проход по столбцам

```

```

        for k in range(mat1.shape[1]): # Суммирование произведений
            result[row, col] += mat1[row, k] * mat2[k, col]
    return result

def difference_matrix(mat1, mat2):
    """
    Вычисление разности двух матриц.
    """
    return mat1 - mat2 # Простое вычитание матриц

def iterative_method(coeff, const):
    """
    Решение СЛАУ методом простых итераций.
    """
    tolerance = 1e-5 # Допустимая погрешность
    last_solution = np.zeros(len(const)) # Начальное приближение
    current_solution = np.ones(len(const)) # Текущее решение

    max_diff = np.max(np.abs(last_solution - current_solution)) # Начальная разница
    iterations = 0 # Счетчик итераций

    while max_diff > tolerance: # Итерационный процесс
        for i in range(coeff.shape[0]):
            # Обновление решения для каждой переменной
            current_solution[i] = const[i] + np.sum(coeff[i, :] * last_solution)

            # Проверка сходимости
            max_diff = np.max(np.abs(last_solution - current_solution))
            last_solution = np.copy(current_solution) # Обновление предыдущего решения
            iterations += 1 # Увеличение счетчика

    return current_solution, iterations

def gauss_seidel_method(coeff, const):
    """
    Решение СЛАУ методом Гаусса-Зейделя.
    """
    tolerance = 1e-4 # Допустимая погрешность
    last_solution = np.zeros(len(const)) # Начальное приближение
    current_solution = np.ones(len(const)) # Текущее решение

    # Разложение матрицы на верхнюю и нижнюю треугольные
    upper_matrix = np.triu(coeff)
    lower_matrix = np.tril(coeff) - np.diag(np.diag(coeff))

```

```

identity = np.eye(len(const)) # Единичная матрица
# Вычисление  $(I - U)^{-1}$ 
delta_inv = compute_inverse(identity - upper_matrix)

# Вычисление произведения  $(I - U)^{-1} * L$ 
prod = matrix_product(delta_inv, lower_matrix)

max_diff = np.max(np.abs(last_solution - current_solution))
iterations = 0

while max_diff > tolerance: # Итерационный процесс
    for i in range(coeff.shape[0]):
        # Обновление решения с использованием уже вычисленных значений
        current_solution[i] = np.sum(prod[i, :] * last_solution) + delta_inv[i,
:] .dot(const)

    max_diff = np.max(np.abs(last_solution - current_solution))
    last_solution = np.copy(current_solution)
    iterations += 1

return current_solution, iterations

def analyze_solution(mat):
    """
    Анализ решения СЛАУ разными методами и сравнение их эффективности.
    """
    # Нормализация матрицы
    normalized_mat = normalize_matrix(mat)
    # Разделение на матрицу коэффициентов и вектор свободных членов
    coeff_matrix = normalized_mat[:, :-1]
    const_vector = normalized_mat[:, -1]

    # Решение обоими методами
    simple_iterations = iterative_method(coeff_matrix, const_vector)
    seidel_iterations = gauss_seidel_method(coeff_matrix, const_vector)

    # Вывод результатов
    print("Результаты метода простых итераций:\n", simple_iterations[0])
    print("Результаты метода Зейделя:\n", seidel_iterations[0])

    # Сравнение количества итераций
    if simple_iterations[1] < seidel_iterations[1]:
        diff = seidel_iterations[1] - simple_iterations[1]

```

```

        print(f"Метод простых итераций потребовал {simple_iterations[1]} итераций, а
метод Зейделя - {seidel_iterations[1]} итераций.\nРазница составляет {diff}
итераций меньше.")
    elif simple_iterations[1] > seidel_iterations[1]:
        diff = simple_iterations[1] - seidel_iterations[1]
        print(f"Метод простых итераций потребовал {simple_iterations[1]} итераций, а
метод Зейделя - {seidel_iterations[1]} итераций.\nРазница составляет {diff}
итераций больше.")
    else:
        print(f"Оба метода потребовали {simple_iterations[1]} итераций, значит они
работают одинаково!")

# Пример использования функции анализа с заданной матрицей
input_matrix = np.array([[10.0, -1.0, -2.0, 5.0, -99.0],
                          [4.0, 28.0, 7.0, 9.0, 0.0],
                          [6.0, 5.0, -23.0, 4.0, 67.0],
                          [1.0, 4.0, 5.0, -15.0, 58.0]])
analyze_solution(input_matrix)

```

Вывод:

Результаты метода простых итераций:

[-7.99999844 4.00000502 -5.0000035 -5.00000364]

Результаты метода Зейделя:

[-8.08530301 3.83319549 -5.07137244 -5.07396306]

Метод простых итераций потребовал 18 итераций, а метод Зейделя - 10 итераций.

Разница составляет 8 итераций больше.

1.4. Реализовать метод вращений в виде программы, задавая в качестве входных данных матрицу и точность вычислений. Используя разработанное программное обеспечение, найти собственные значения и собственные векторы симметрических матриц. Проанализировать зависимость погрешности вычислений от числа итераций.

$$\begin{pmatrix} -7 & -9 & 1 \\ -9 & 7 & 2 \\ 1 & 2 & 9 \end{pmatrix}$$

Код:

```

import numpy as np

# Функция для нахождения индексов максимального по модулю элемента в матрице A
def max_modul_element_of_matrix(A):
    max1 = 0 # Инициализируем максимальный элемент
    for i in range(A.shape[0]): # Проходим по строкам
        for j in range(A.shape[1]): # Проходим по столбцам
            # Проверяем, что i не равно j и элемент больше текущего максимума
            if i != j and max1 < abs(A[i, j]):
                max1 = abs(A[i, j]) # Обновляем максимум
                i1 = i # Сохраняем индекс строки

```

```

        j1 = j # Сохраняем индекс столбца
    return i1, j1 # Возвращаем индексы максимального элемента

# Функция для транспонирования матрицы A
def transpose(A):
    AT = np.zeros((A.shape[0], A.shape[1])) # Создаем матрицу для транспонирования
    for i in range(A.shape[0]): # Проходим по строкам
        for j in range(A.shape[1]): # Проходим по столбцам
            AT[j, i] = A[i, j] # Заполняем транспонированную матрицу
    return AT # Возвращаем транспонированную матрицу

# Функция для умножения матриц A и B
def multiply(A, B):
    C = np.zeros((A.shape[0], B.shape[1])) # Создаем матрицу для результата
    for i in range(C.shape[0]): # Проходим по строкам результата
        for j in range(C.shape[1]): # Проходим по столбцам результата
            for k in range(A.shape[1]): # Умножаем соответствующие элементы
                C[i, j] += A[i, k] * B[k, j] # Обновляем элемент результирующей
матрицы
    return C # Возвращаем результирующую матрицу

# Функция для нахождения собственных значений и векторов методом вращений
def rotation_method(A):
    m = 3 # Инициализируем начальное значение для m
    e = 0.003 # Задаем значение точности
    U = [] # Список для хранения вращательных матриц
    A1 = np.copy(A) # Создаем копию исходной матрицы A
    while m > e: # Продолжаем, пока m больше точности e
        m = 0 # Сброс значения m
        A0 = np.copy(A1) # Создаем копию текущей матрицы
        iMax = max_modul_element_of_matrix(A0)[0] # Находим индекс строки с
максимальным элементом
        jMax = max_modul_element_of_matrix(A0)[1] # Находим индекс столбца с
максимальным элементом
        # Вычисляем угол вращения phi
        if A0[iMax, iMax] != A0[jMax, jMax]:
            phi = np.arctan(2 * A0[iMax, jMax] / (A0[iMax, iMax] - A0[jMax, jMax]))
/ 2
        else:
            phi = np.pi / 4 # Если делим на ноль, задаем стандартный угол
        U0 = np.zeros((A0.shape[0], A0.shape[1])) # Создаем вращательную матрицу
        for i in range(U0.shape[0]):

```

```

        for j in range(U0.shape[1]):
            # Заполняем вращательную матрицу значениями
            if i == iMax and j == jMax:
                U0[i, j] = -np.sin(phi)
                U0[i, i] = np.cos(phi)
                U0[j, j] = np.cos(phi)
                U0[j, i] = np.sin(phi)
            elif i == j and i != iMax and j != jMax:
                U0[i, j] = 1

    U.append(U0) # Сохраняем текущую матрицу в список
    U0T = transpose(U0) # Транспонируем текущую матрицу
    A1 = multiply(multiply(U0T, A0), U0) # Обновляем матрицу A1
    for i in range(A1.shape[0]): # Вычисляем новое значение m
        for j in range(A1.shape[1]):
            if i < j:
                m += A1[i, j] * A1[i, j] # Подсчитываем сумму квадратов над
диагональными элементами
        m = np.sqrt(m) # Извлекаем корень из суммы

    # Печатаем собственные значения
    for i in range(A1.shape[0]):
        for j in range(A1.shape[1]):
            if i == j:
                print("Собственное значение №", i + 1, np.around(A1[i, j], 2))

    # Вычисляем собственные векторы
    for i in range(len(U) - 1):
        U[i + 1] = multiply(U[i], U[i + 1])

    U = U[-1] # Получаем последний результат
    for i in range(U.shape[0]): # Печатаем собственные векторы
        print("Собственный вектор №", i + 1)
        for j in range(U.shape[1]):
            print(np.around(U[j, i], 2), sep='\n')

    return np.around(A1, 2) # Возвращаем матрицу собственных значений

# Главная функция для обработки
def main(A):
    print("Матрица A:\n", np.around(rotation_method(A), 2)) # Печатаем обработанную
матрицу

# Исходная матрица
A = np.array([[-7, -9, 1],

```

```

        [-9, 7, 2],
        [1, 2, 9]])
main(A) # Запускаем главную функцию

```

Вывод:

Собственное значение № 1 -11.56

Собственное значение № 2 12.04

Собственное значение № 3 8.52

Собственный вектор № 1

0.89

0.44

-0.09

Собственный вектор № 2

-0.37

0.83

0.42

Собственный вектор № 3

0.26

-0.35

0.9

Матрица A:

$\begin{bmatrix} -11.56 & 0. & -0. \end{bmatrix}$

$\begin{bmatrix} 0. & 12.04 & -0. \end{bmatrix}$

$\begin{bmatrix} 0. & -0. & 8.52 \end{bmatrix}$

1.5. Реализовать алгоритм QR – разложения матриц в виде программы. На его основе разработать программу, реализующую QR – алгоритм решения полной проблемы собственных значений произвольных матриц, задавая в качестве входных данных матрицу и точность вычислений. С использованием разработанного программного обеспечения найти собственные значения матрицы.

$$\begin{pmatrix} 6 & 5 & -6 \\ 4 & -6 & 9 \\ -6 & 6 & 1 \end{pmatrix}$$

Код:

```

import numpy as np

def sign(x):
    """Определяет знак числа. Используется при построении матриц Хаусхолдера для
    обеспечения численной устойчивости."""
    if x > 0:
        return 1
    if x < 0:
        return -1
    return 0

def multiply(A, B):

```

```

"""Умножение матриц A и B. Умножение Q R """
# Проверка совместимости размеров матриц
if A.shape[1] != B.shape[0]:
    raise ValueError("Несовместимые размеры матриц для умножения")

# Создаем результирующую матрицу
C = np.zeros((A.shape[0], B.shape[1]))
for i in range(A.shape[0]):
    for j in range(B.shape[1]):
        # Вычисляем элемент C[i,j] как сумму произведений элементов строки A и
        # столбца B
        for k in range(A.shape[1]):
            C[i,j] += A[i,k] * B[k,j]
    return C

def division(A, B):
    """Деление элементов матрицы A на скаляр B. Используется при нормировке
    векторов."""
    C = np.zeros((A.shape[0], A.shape[1]))
    for i in range(C.shape[0]):
        for j in range(C.shape[1]):
            C[i,j] = A[i,j] / B # Каждый элемент матрицы делится на скаляр
    return C

def difference(A, B):
    """Разность матриц A и B, вычитает из единичной матрицы полученную"""
    # Проверка одинаковости размеров матриц
    if A.shape != B.shape:
        raise ValueError("Матрицы должны быть одного размера")

    C = np.zeros((A.shape[0], A.shape[1]))
    for i in range(C.shape[0]):
        for j in range(C.shape[1]):
            C[i,j] = A[i,j] - B[i,j] # Поэлементное вычитание
    return C

def norm_vector(b):
    """Вычисление евклидовой нормы вектора"""
    norm = 0
    for i in range(len(b)):
        norm += b[i]*b[i] # Сумма квадратов компонент
    return np.sqrt(norm) # Квадратный корень из суммы квадратов

def Householder_matrix(H, index):

```

```

        """Построение матрицы Хаусхолдера для QR-разложения. строит матрицу
преобразования Хаусхолдера"""
        H1 = np.copy(H)
        n = H1.shape[0]

        # Формирование вектора для преобразования
        x = np.zeros(n)
        for i in range(n):
            if i < index:
                x[i] = 0 # Элементы выше индекса остаются нулевыми
            elif i == index:
                # Главный элемент: добавляем знак нормы для устойчивости
                x[i] = H1[i,index] + sign(H1[i,index]) * norm_vector(H1[index:,index])
            else:
                x[i] = H1[i,index] # Остальные элементы берутся из столбца

        # Нормализация вектора
        norm_x = norm_vector(x)
        if norm_x == 0:
            return np.eye(n) # Если вектор нулевой, возвращаем единичную матрицу

        x = x / norm_x # Нормировка вектора

        # Построение матрицы Хаусхолдера:  $H = I - 2xx^T$ 
        E = np.eye(n)
        H = difference(E, 2 * np.outer(x, x)) # Внешнее произведение вектора на себя
        return H

def QR(A):
    """QR-разложение матрицы методом Хаусхолдера, чтобы получить: R (верхняя
треугольная матрица), Q (ортогональная матрица)
    """
    A0 = np.copy(A)
    n = A0.shape[0]
    Q = np.eye(n) # Начинаем с единичной матрицы

    for i in range(n - 1):
        # Построение матрицы Хаусхолдера для текущего столбца
        H = Householder_matrix(A0, i)
        # Применение преобразования Хаусхолдера
        A0 = multiply(H, A0)
        Q = multiply(Q, H) # Накопление преобразований в Q

    return A0, Q # Возвращаем R (A0) и Q

```

```

def eigenvalues(A, epsilon=1e-6, max_iter=1000):
    """Вычисление собственных значений методом QR-алгоритма"""
    A0 = np.copy(A)
    n = A0.shape[0]

    for _ in range(max_iter):
        R, Q = QR(A0)
        A0 = multiply(R, Q) # Обновление матрицы A0 = RQ

        # Проверка на сходимость (нижний треугольник близок к нулю)
        off_diag_norm = 0
        for i in range(n):
            for j in range(i):
                off_diag_norm += A0[i,j]**2 # Сумма квадратов поддиагональных
элементов

        # Если норма поддиагонали меньше epsilon, прерываем итерации
        if np.sqrt(off_diag_norm) < epsilon:
            break

    if n == 3:
        # Проверяем, достаточно ли малы поддиагональные элементы первого столбца
        if abs(A0[1,0]) < epsilon and abs(A0[2,0]) < epsilon:
            # Первое собственное значение - диагональный элемент
            lambda1 = A0[0,0]

            # Для второго и третьего собственных значений анализируем блок 2x2
            a22 = A0[1,1]
            a23 = A0[1,2]
            a32 = A0[2,1]
            a33 = A0[2,2]

            # Решаем квадратное уравнение для блока 2x2
            discriminant = (a22 + a33)**2 - 4*(a22*a33 - a23*a32)

            if discriminant < 0:
                # Комплексно-сопряженные корни
                real_part = (a22 + a33) / 2
                imag_part = np.sqrt(-discriminant) / 2
                lambda2 = complex(real_part, imag_part)
                lambda3 = complex(real_part, -imag_part)

            # Округляем до двух знаков после запятой, как в примере
            lambda1 = round(lambda1, 2)
            lambda2 = complex(round(lambda2.real, 2), round(lambda2.imag, 2))

```

```

        lambda3 = complex(round(lambda3.real, 2), round(lambda3.imag, 2))

    return [lambda1, lambda2, lambda3]

elif discriminant > 0:
    # Два различных действительных корня
    sqrt_discr = np.sqrt(discriminant)
    lambda2 = (a22 + a33 + sqrt_discr) / 2
    lambda3 = (a22 + a33 - sqrt_discr) / 2

    # Округляем до двух знаков после запятой
    lambda1 = round(lambda1, 2)
    lambda2 = round(lambda2, 2)
    lambda3 = round(lambda3, 2)

    return [lambda1, lambda2, lambda3]
else:
    # Кратный действительный корень
    lambda2 = (a22 + a33) / 2
    lambda3 = lambda2

    # Округляем до двух знаков после запятой
    lambda1 = round(lambda1, 2)
    lambda2 = round(lambda2, 2)
    lambda3 = round(lambda3, 2)

    return [lambda1, lambda2, lambda3]

# Если не выполнились специальные условия, возвращаем диагональные элементы
return [round(A0[i,i], 2) for i in range(n)]

def main(A):
    """Основная функция для демонстрации работы"""
    # Выполняем QR-разложение
    R, Q = QR(A) # Обратите внимание: возвращается (R, Q)

    # Вычисляем собственные значения
    eigvals = eigenvalues(A)

    # Вывод результатов
    print("Исходная матрица A:\n", A)
    print("\nМатрица Q:\n", Q)
    print("\nМатрица R:\n", R)
    print("\nСобственные значения матрицы A:\n", eigvals)

```

```

if __name__ == "__main__":
    # Пример матрицы для демонстрации
    A = np.array([[6.0, 5.0, -6.0],
                  [4.0, -6.0, 9.0],
                  [-6.0, 6.0, 1.0]])

    main(A)

```

Вывод:

Исходная матрица A:

```

[[ 6.  5. -6.]
 [ 4. -6.  9.]
 [-6.  6.  1.]]

```

Матрица Q:

```

[[-0.63960215  0.75634056  0.13732458]
 [-0.42640143 -0.49772089  0.7552852 ]
 [ 0.63960215  0.42452664  0.64084805]]

```

Матрица R:

```

[[-9.38083152e+00  3.19801075e+00  6.39602149e-01]
 [ 2.28046293e-15  9.31518799e+00 -8.59300474e+00]
 [ 1.94982769e-16  8.88178420e-16  6.61446737e+00]]

```

Собственные значения матрицы A:

```

[np.float64(-13.28), np.float64(9.86), np.float64(4.41)]

```

Лабораторная работа №2

Вариант 20

2.1. Реализовать методы простой итерации и Ньютона решения нелинейных уравнений в виде программ, задавая в качестве входных данных точность вычислений. С использованием разработанного программного обеспечения найти положительный корень нелинейного уравнения (начальное приближение определить графически). Проанализировать зависимость погрешности вычислений от количества итераций.

$$\operatorname{tg} x - 5x^2 + 1 = 0, \quad x \in [-1, 1]$$

Код: `import numpy as np`
`import matplotlib.pyplot as plt`

```

def plot_function_f(x):
    """
    Функция для построения графиков двух функций:
    1. y = tg(x)
    2. y = 5x^2 - 1
    """

```

```

    Возвращает кортеж значений обеих функций в точках x
    """
    return np.tan(x), 5 * x**2 - 1

def function_fi(x):
    """
    Функция для метода простых итераций.
    Преобразованное уравнение:  $x = \arctg(5x^2 - 1)$ 
    """
    return np.arctan(5 * x**2 - 1)

def function_f(x):
    """
    Исходная функция  $f(x) = \operatorname{tg}(x) - 5x^2 + 1$ 
    Уравнение, которое нужно решить:  $f(x) = 0$ 
    """
    return np.tan(x) - 5 * x**2 + 1

def derivative_f(x):
    """
    Производная исходной функции:  $f'(x) = 1/\cos^2(x) - 10x$ 
    Используется в методе Ньютона
    """
    return 1/(np.cos(x)**2) - 10 * x

def second_derivative_f(x):
    """
    Вторая производная исходной функции:  $f''(x) = 2*\sin(x)/\cos^3(x) - 10$ 
    Используется для проверки условия сходимости метода Ньютона
    """
    return 2 * np.sin(x)/(np.cos(x)**3) - 10

def simple_iter(e, q, a, b):
    """
    Метод простых итераций для нахождения корня уравнения
    Параметры:
    e - точность вычислений
    q - константа Липшица ( $0 < q < 1$ )
    a, b - границы интервала поиска
    """
    x_prev = (a + b) / 2 # Начальное приближение
    x_next = function_fi(x_prev) # Следующее приближение

    while (q / (1 - q)) * abs(x_next - x_prev) > e:
        x_prev = x_next

```

```

        x_next = function_fi(x_prev)

    return x_next

def newton_method(e, a, b):
    """
    Метод Ньютона (метод касательных) для нахождения корня
    Параметры:
    e - точность вычислений
    a, b - границы интервала поиска
    """
    if function_f(a) * function_f(b) >= 0:
        raise ValueError("Функция не меняет знак на концах интервала [a, b]")

    x_prev = None
    for x_candidate in [a, b, (a+b)/2]:
        if function_f(x_candidate) * second_derivative_f(x_candidate) > 0:
            x_prev = x_candidate
            break

    if x_prev is None:
        raise ValueError("Не удалось найти начальное приближение, удовлетворяющее условию сходимости")

    x_next = x_prev - function_f(x_prev) / derivative_f(x_prev)

    while abs(x_next - x_prev) > e:
        x_prev = x_next
        x_next = x_prev - function_f(x_prev) / derivative_f(x_prev)

    return x_next

def main():
    e = 0.0001 # Точность вычислений
    q = 0.2    # Константа Липшица для метода простых итераций

    # Настройки графика
    plt.title("График тангенса и параболы")
    plt.xlabel("x")
    plt.ylabel("y")
    plt.grid()
    plt.axis([-1, 1, -5, 5]) # Изменённые границы для лучшего отображения

    # Построение графиков функций
    x = np.linspace(-1, 1, 1000) # Более плотная сетка для тангенса

```

```

y = plot_function_f(x)
plt.plot(x, y[0], label='tg(x)')
plt.plot(x, y[1], label='5x2-1')
plt.legend()
plt.show()

# Используем фиксированный интервал [-1, 1] как в условии
a, b = -1, 1

try:
    print("\nМетодом простых итераций:")
    print(f"Корень уравнения на промежутке от {a} до {b} является
{np.around(simple_iter(e, q, a, b), 5)}")

    print("\nМетодом Ньютона:")
    print(f"Корень уравнения на промежутке от {a} до {b} является
{np.around(newton_method(e, a, b), 5)}")
except ValueError as e:
    print(f"\nОшибка: {e}")

if __name__ == "__main__":
    main()

```

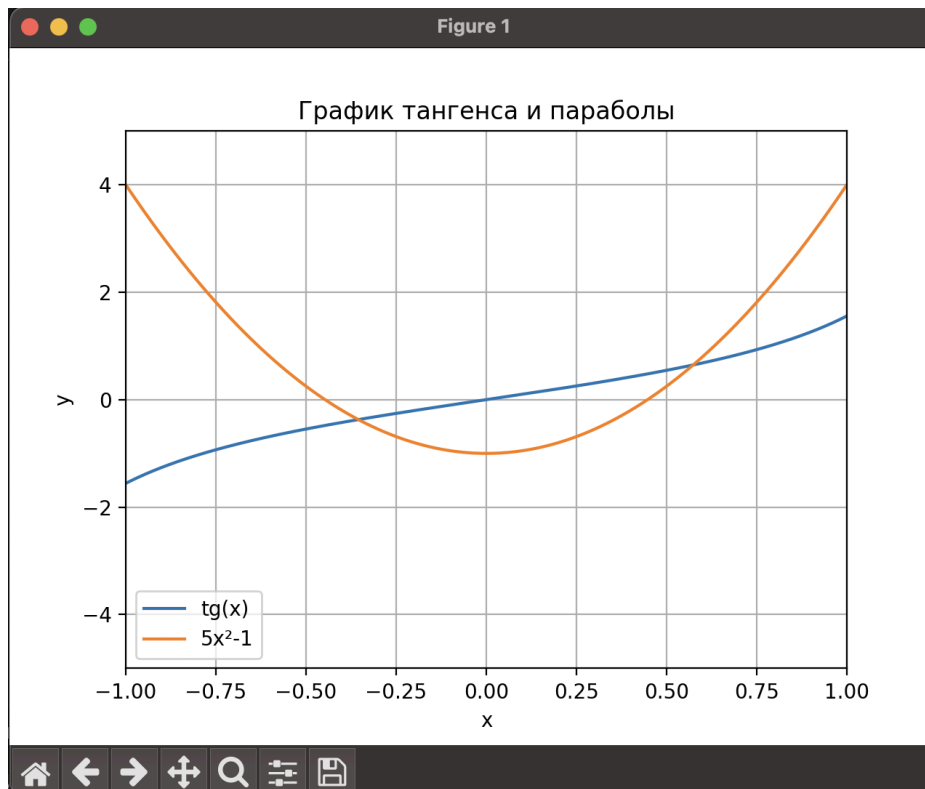
Вывод:

Методом простых итераций:

Корень уравнения на промежутке от -1 до 1 является 1.46895

Методом Ньютона:

Корень уравнения на промежутке от -1 до 1 является 1.46895



2.2. Реализовать методы простой итерации и Ньютона решения систем нелинейных уравнений в виде программного кода, задавая в качестве входных данных точность вычислений. С использованием разработанного программного обеспечения решить систему нелинейных уравнений (при наличии нескольких решений найти то из них, в котором значения неизвестных являются положительными); начальное приближение определить графически. Проанализировать зависимость погрешности вычислений от количества итераций.

20	2
----	---

$$\begin{cases} x_1^2 - 2\lg x_2 - 1 = 0, \\ x_1^2 - ax_1x_2 + a = 0. \end{cases}$$

Код:

```
import numpy as np
import matplotlib.pyplot as plt
from math import log10

# Определение функций системы уравнений и их производных
def function_fi(x1, x2):
    """Функция итерационного процесса (правая часть системы уравнений)"""
    return np.sqrt(2 * log10(x2) + 1), (x1**2 + 2) / (2 * x1)

def function_f(x1, x2):
    """Исходная система уравнений (левая часть)"""
```

```

    return x1**2 - 2 * log10(x2) - 1, x1**2 - 2 * x1 * x2 + 2

# Частные производные для матрицы Якоби
def df1x1(x1, x2):
    """Производная первого уравнения по x1"""
    return 2 * x1

def df1x2(x1, x2):
    """Производная первого уравнения по x2"""
    return -2 / (x2 * np.log(10))

def df2x1(x1, x2):
    """Производная второго уравнения по x1"""
    return 2 * x1 - 2 * x2

def df2x2(x1, x2):
    """Производная второго уравнения по x2"""
    return -2 * x1

# Определители для метода Ньютона
def detA1(x1, x2):
    """Определитель матрицы A1 (замена первого столбца матрицы Якоби на вектор
F)"""
    f1, f2 = function_f(x1, x2)
    return f1 * df2x2(x1, x2) - f2 * df1x2(x1, x2)

def detA2(x1, x2):
    """Определитель матрицы A2 (замена второго столбца матрицы Якоби на вектор
F)"""
    f1, f2 = function_f(x1, x2)
    return df1x1(x1, x2) * f2 - df2x1(x1, x2) * f1

def detJ(x1, x2):
    """Определитель матрицы Якоби J"""
    return df1x1(x1, x2) * df2x2(x1, x2) - df2x1(x1, x2) * df1x2(x1, x2)

def norm(b):
    """Вычисление максимальной нормы вектора"""
    return max(abs(b[0]), abs(b[1]))

def simple_iter(e, q, a, b):
    """
    Метод простых итераций для решения системы уравнений
    Параметры:
    e - требуемая точность

```

```

q - параметр сходимости
a, b - границы начального приближения
"""

# Начальные приближения
x1_prev, x2_prev = (a + b)/2, (a + b)/2
x1_next, x2_next = function_fi(x1_prev, x2_prev)

# Итерационный процесс
while q / (1 - q) * max(abs(x1_next - x1_prev), abs(x2_next - x2_prev)) >
e:

    x1_prev, x2_prev = x1_next, x2_next
    x1_next, x2_next = function_fi(x1_prev, x2_prev)

return [x1_next, x2_next]

def newton_method(e, a, b):
    """
    Метод Ньютона для решения системы уравнений
    Параметры:
    e - требуемая точность
    a, b - границы начального приближения
    """

    # Начальные приближения
    x1, x2 = (a + b)/2, (a + b)/2

    while True:
        # Вычисление поправок
        delta_x1 = detA1(x1, x2) / detJ(x1, x2)
        delta_x2 = detA2(x1, x2) / detJ(x1, x2)

        # Обновление приближений
        x1_new = x1 - delta_x1
        x2_new = x2 - delta_x2

        # Проверка условия выхода
        if max(abs(x1_new - x1), abs(x2_new - x2)) < e:
            break

        x1, x2 = x1_new, x2_new

    return [x1, x2]

def main():
    """Основная функция программы"""
    e = 0.0001 # Точность вычислений

```

```

q = 0.2      # Параметр сходимости для метода простых итераций

# Построение графиков функций
plt.figure(figsize=(10, 5))

# Первое уравнение:  $x_1^2 - 2\lg(x_2) - 1 = 0$ 
plt.subplot(1, 2, 1)
x2 = np.linspace(0.1, 10, 1000)
x1_eq1 = np.sqrt(2 * np.log10(x2) + 1)
plt.plot(x1_eq1, x2, label='x12 - 2lg(x2) - 1 = 0')
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Первое уравнение')
plt.grid()
plt.legend()

# Второе уравнение:  $x_1^2 - 2x_1x_2 + 2 = 0$ 
plt.subplot(1, 2, 2)
x1 = np.linspace(-3, 3, 1000)
x2_eq2 = (x1**2 + 2) / (2 * x1)
plt.plot(x1, x2_eq2, label='x12 - 2x1x2 + 2 = 0')
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Второе уравнение')
plt.grid()
plt.legend()

plt.tight_layout()
plt.show()

# Используем фиксированные начальные приближения
print("Введите начальные приближения для x1 и x2 (через пробел):")
x1_init, x2_init = map(float, input().split())

# Вывод результатов
try:
    print("\nМетодом простых итераций:")
    roots = simple_iter(e, q, x1_init, x2_init)
    print(f"Корни системы уравнений: x1 = {roots[0]:.5f}, x2 = {roots[1]:.5f}")

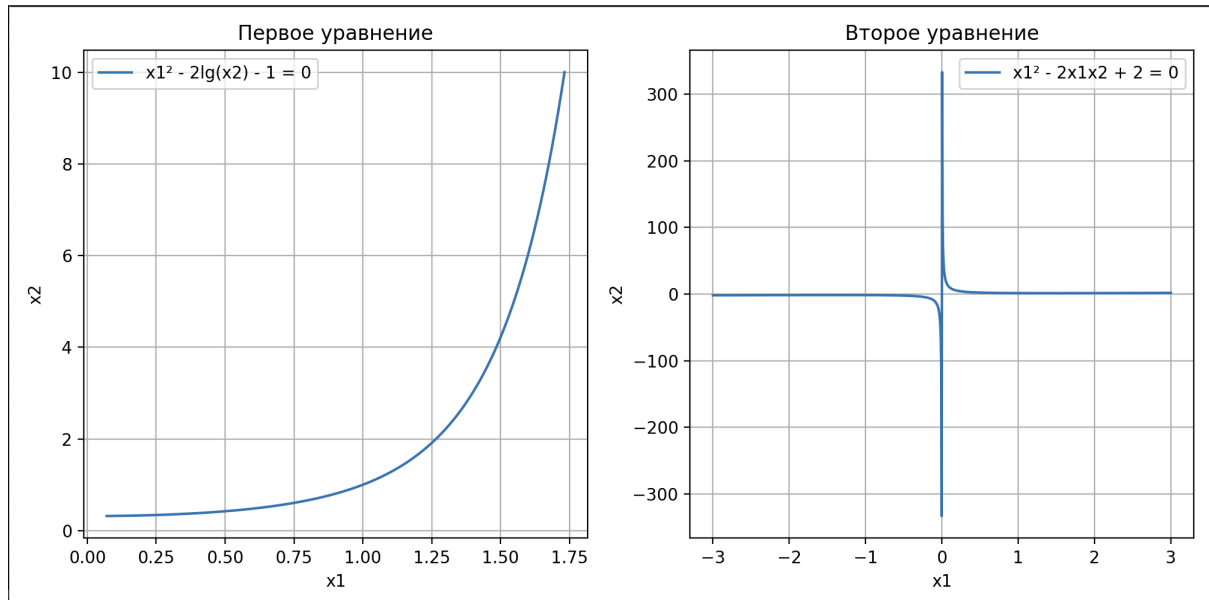
    print("\nМетодом Ньютона:")
    roots = newton_method(e, x1_init, x2_init)
    print(f"Корни системы уравнений: x1 = {roots[0]:.5f}, x2 = {roots[1]:.5f}")

```

```
except Exception as ex:
    print(f"\nОшибка: {ex}")

if __name__ == "__main__":
    main()
```

Вывод:



Лабораторная работа №3 Вариант 20

3.1. Используя таблицу значений Y_i функции $y = f(x)$, вычисленных в точках X_i , $i = 0, \dots, 3$ построить интерполяционные многочлены Лагранжа и Ньютона, проходящие через точки $\{X_i, Y_i\}$. Вычислить значение погрешности интерполяции в точке X^* .

20. $y = \arccos(x) + x$, а) $X_i = -0.4, -0.1, 0.2, 0.5$; б) $X_i = -0.4, 0, 0.2, 0.5$; $X^* = 0.1$.

Код:

```
import numpy as np
import matplotlib.pyplot as plt
from math import factorial

def func(x1, x2):
    """Функция разделенной разности первого порядка"""
    if x1 == x2:
        raise ValueError("x1 and x2 must be different")
    return (np.arctan(x1) - np.arctan(x2)) / (x1 - x2)
```

```

def func1(x1, x2, x3):
    """Функция разделенной разности второго порядка"""
    return (func(x1, x2) - func(x2, x3)) / (x1 - x3) if x1 != x3 else 0.0

def func2(x1, x2, x3, x4):
    """Функция разделенной разности третьего порядка"""
    return (func1(x1, x2, x3) - func1(x2, x3, x4)) / (x1 - x4) if x1 != x4
else 0.0

def couting_w(x):
    """Вычисление знаменателей для базисных полиномов Лагранжа"""
    if len(x) != 4:
        raise ValueError("Expected 4 interpolation nodes")
    w = np.zeros(4)
    for i in range(4):
        product = 1.0
        for j in range(4):
            if j != i:
                product *= (x[i] - x[j])
        w[i] = product
    return w

def Lagrange_method(x, X):
    """Метод интерполяции Лагранжа"""
    if len(x) != 4:
        raise ValueError("Lagrange method requires exactly 4 nodes")

    w = couting_w(x)
    y_values = np.arctan(x)

    L = 0.0
    for i in range(4):
        term = y_values[i] / w[i]
        for j in range(4):
            if j != i:
                term *= (X - x[j])
        L += term

    exact = np.arctan(X)
    delta = abs(exact - L)
    return L, exact, delta

def Newton_method(x, X):
    """Метод интерполяции Ньютона с разделенными разностями"""

```

```

    if len(x) != 4:
        raise ValueError("Newton method requires exactly 4 nodes")

    # Вычисляем разделенные разности
    dd = np.zeros((4, 4))
    dd[0, :] = np.arctan(x)

    for i in range(1, 4): # Заполняем таблицу разностей
        for j in range(4 - i):
            dd[i, j] = (dd[i-1, j] - dd[i-1, j+1]) / (x[j] - x[j+1]) if x[j]
!= x[j+1] else 0.0

    # Вычисляем интерполяционный полином
    P = dd[0, 0]
    product = 1.0
    for i in range(1, 4):
        product *= (X - x[i-1])
        P += dd[i, 0] * product

    exact = np.arctan(X)
    delta = abs(exact - P)
    return P, exact, delta

def error_estimation(x, X, n=3):
    """
    Оценка погрешности интерполяции
     $|\varepsilon_n(x)| \leq M_{\{n+1\}} / (n+1)! \cdot |\omega_{\{n+1\}}(x)|$ 
    """
    if len(x) != n+1:
        raise ValueError(f"Need exactly {n+1} nodes for degree {n}
interpolation")

    # Вычисляем  $\omega_{\{n+1\}}(x)$ 
    omega = 1.0
    for xi in x:
        omega *= (X - xi)

    # Вычисляем  $M_{\{n+1\}}$  - максимум (n+1)-й производной на интервале
    interval = np.linspace(min(x), max(x), 1000)

    if n == 3: # Для кубической интерполяции (n=3)
        # Четвертая производная arctan(x):  $(-24x^3 + 24x) / (1+x^2)^4$ 
        fourth_deriv = (-24*interval**3 + 24*interval) / (1 + interval**2)**4
        M = max(abs(fourth_deriv))
    else:

```

```

        raise NotImplementedError("Only n=3 is implemented")

    error_bound = (M / factorial(n+1)) * abs(omega)
    return error_bound

def plot_interpolation(x_nodes, title):
    """Функция для построения графиков интерполяции"""
    x_vals = np.linspace(min(x_nodes)-0.5, max(x_nodes)+0.5, 400)
    y_true = np.arctan(x_vals)

    # Вычисляем значения интерполяционных полиномов
    y_lagrange = np.zeros_like(x_vals)
    y_newton = np.zeros_like(x_vals)

    for i, x in enumerate(x_vals):
        L, _, _ = Lagrange_method(x_nodes, x)
        P, _, _ = Newton_method(x_nodes, x)
        y_lagrange[i] = L
        y_newton[i] = P

    plt.figure(figsize=(12, 6))
    plt.plot(x_vals, y_true, label='arctan(x)', linewidth=2)
    plt.plot(x_vals, y_lagrange, '--', label='Lagrange interpolation')
    plt.plot(x_vals, y_newton, '-.', label='Newton interpolation')
    plt.scatter(x_nodes, np.arctan(x_nodes), color='red', zorder=5,
label='Interpolation nodes')

    plt.title(title)
    plt.xlabel('x')
    plt.ylabel('y')
    plt.legend()
    plt.grid(True)
    plt.show()

def calculate_errors(x_nodes, point):
    """Вычисление погрешностей интерполяции в заданной точке"""
    # Фактическая погрешность
    _, exact, delta_lagrange = Lagrange_method(x_nodes, point)
    _, _, delta_newton = Newton_method(x_nodes, point)

    # Теоретическая оценка погрешности
    error_bound = error_estimation(x_nodes, point)

    return {
        'exact_value': exact,
        'lagrange_error': delta_lagrange,

```

```

        'newton_error': delta_newton,
        'error_bound': error_bound
    }

def main(x1, x2, root):
    """Основная функция для сравнения методов интерполяции"""
    try:
        # Проверка входных данных
        if len(x1) != 4 or len(x2) != 4:
            raise ValueError("Both node sets must contain exactly 4 points")

        # Вычисления для первого набора узлов
        L1, Ly1, Ldelta1 = Lagrange_method(x1, root)
        P1, Py1, Pdelta1 = Newton_method(x1, root)
        error_bound1 = error_estimation(x1, root)

        # Вычисления для второго набора узлов
        L2, Ly2, Ldelta2 = Lagrange_method(x2, root)
        P2, Py2, Pdelta2 = Newton_method(x2, root)
        error_bound2 = error_estimation(x2, root)

        # Вывод результатов
        print("\nResults for node set 1:", x1)
        print(f"Lagrange: L({root}) = {L1:.6f}, exact = {Ly1:.6f}, error = {Ldelta1:.6f}, bound = {error_bound1:.6f}")
        print(f"Newton: P({root}) = {P1:.6f}, exact = {Py1:.6f}, error = {Pdelta1:.6f}, bound = {error_bound1:.6f}")

        print("\nResults for node set 2:", x2)
        print(f"Lagrange: L({root}) = {L2:.6f}, exact = {Ly2:.6f}, error = {Ldelta2:.6f}, bound = {error_bound2:.6f}")
        print(f"Newton: P({root}) = {P2:.6f}, exact = {Py2:.6f}, error = {Pdelta2:.6f}, bound = {error_bound2:.6f}")

        # Дополнительный вывод погрешности в формате, аналогичном примеру из
        изображения
        print("\n" + "="*50)
        print("Детальный анализ погрешности для x1:")
        print(f"L3({root}) = {L1:.5f}, y({root}) = {Ly1:.5f}")
        print(f"Абсолютная погрешность интерполяции:  $\Delta$  = {Ldelta1:.5f}")
        print(f"Теоретическая оценка погрешности: {error_bound1:.5f}")

        # Построение графиков
        plot_interpolation(x1, "Interpolation for node set 1: " + str(x1))

```

```

except Exception as e:
    print(f"Error: {str(e)}")

# Два разных набора узлов интерполяции
x1 = np.array([-4.0, -1.0, 0.2, 0.5]) # Равномерно распределенные узлы
x2 = np.array([-4.0, 0.0, 0.2, 0.5]) # Неравномерно распределенные узлы

# Точка, в которой вычисляется интерполяция
root = -0.5

# Запуск основной функции
main(x1, x2, root)

```

Вывод:

Results for node set 1: [-4. 0. 0.2 0.5]

Lagrange: $L(-0.5) = -0.535506$, exact = -0.463648, error = 0.071858, bound = 0.238290

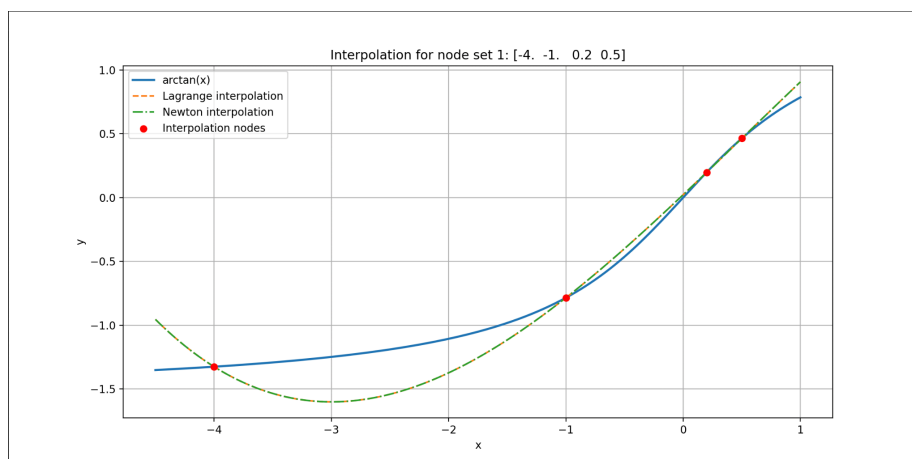
Newton: $P(-0.5) = -0.535506$, exact = -0.463648, error = 0.071858, bound = 0.238290

Детальный анализ погрешности для x1:

$L3(-0.5) = -0.40017$, $y(-0.5) = -0.46365$

Абсолютная погрешность интерполяции: $\Delta = 0.06348$

Теоретическая оценка погрешности: 0.23829



3.2. Построить кубический сплайн для функции, заданной в узлах интерполяции,

предполагая, что сплайн имеет нулевую кривизну при $x = x_0$ и $x = x_4$. Вычислить значение функции в точке $x = X^*$.

20. $X^* = 0.1$

i	0	1	2	3	4
x_i	-0.4	-0.1	0.2	0.5	0.8
f_i	1.5823	1.5710	1.5694	1.5472	1.4435

Код:

```
import numpy as np
import matplotlib.pyplot as plt

def Cubic_spline(x, y, X):
    """
    Функция для вычисления значения кубического сплайна в точке X
    Параметры:
    x - массив узлов интерполяции
    y - массив значений функции в узлах
    X - точка, в которой вычисляется значение сплайна
    """
    n = len(x) - 1 # Количество интервалов

    # Инициализация матриц для системы уравнений
    A = np.zeros((n+1, n+1))
    B = np.zeros(n+1)

    # Заполнение матрицы системы уравнений для вторых производных
    for i in range(1, n):
        h_prev = x[i] - x[i-1]
        h_next = x[i+1] - x[i]

        A[i, i-1] = h_prev
        A[i, i] = 2 * (h_prev + h_next)
        A[i, i+1] = h_next

        B[i] = 3 * ((y[i+1] - y[i]) / h_next - (y[i] - y[i-1]) / h_prev) #di

    # Натуральные краевые условия (M_0 = M_n = 0)
    A[0, 0] = 1
    A[n, n] = 1

    # Решение системы для вторых производных
    M = np.linalg.solve(A, B)

    # Вычисление коэффициентов сплайнов для каждого отрезка
    coeffs = np.zeros((n, 4))
    for i in range(n):
        h = x[i+1] - x[i]
        coeffs[i, 0] = y[i] # a_i
```

```

        coeffs[i, 1] = (y[i+1] - y[i]) / h - h * (2*M[i] + M[i+1]) / 3 # b_i
        coeffs[i, 2] = M[i] # c_i
        coeffs[i, 3] = (M[i+1] - M[i]) / (3 * h) # d_i

# Определение, на каком отрезке находится X
segment = np.searchsorted(x, X) - 1
segment = max(0, min(segment, n-1))

# Вычисление значения сплайна в точке X
dx = X - x[segment]
f = (coeffs[segment, 0] +
     coeffs[segment, 1] * dx +
     coeffs[segment, 2] * dx**2 +
     coeffs[segment, 3] * dx**3)

return f, coeffs

def plot_spline(x, y, spline_coeffs, X=None):
    """Функция для построения графика сплайна"""
    plt.figure(figsize=(10, 6))

    # Построение исходных точек
    plt.plot(x, y, 'o', label='Узлы интерполяции', markersize=8)

    # Построение сплайна для каждого отрезка
    for i in range(len(x)-1):
        x_vals = np.linspace(x[i], x[i+1], 100)
        dx = x_vals - x[i]
        y_vals = (spline_coeffs[i, 0] +
                  spline_coeffs[i, 1] * dx +
                  spline_coeffs[i, 2] * dx**2 +
                  spline_coeffs[i, 3] * dx**3)
        plt.plot(x_vals, y_vals, label=f'S_{i}(x)')

    # Отображение точки X, если задана
    if X is not None:
        segment = np.searchsorted(x, X) - 1
        segment = max(0, min(segment, len(x)-2))
        dx = X - x[segment]
        Y = (spline_coeffs[segment, 0] +
              spline_coeffs[segment, 1] * dx +
              spline_coeffs[segment, 2] * dx**2 +
              spline_coeffs[segment, 3] * dx**3)
        plt.plot(X, Y, 'ro', label=f'Точка X ({X:.2f}, {Y:.4f})')

```

```

plt.title('Кубический сплайн с естественными краевыми условиями')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True)
plt.show()

def main(x, y, X):
    """
    Основная функция для демонстрации работы кубического сплайна
    """
    print("Заданный X =", X, "находится в этом отрезке:", x[1], "-", x[2])
    print("Значения функции на этом отрезке:", y[1], "-", y[2])
    f_value, coeffs = Cubic_spline(x, y, X)
    print("Значение функции в X методом кубического сплайна:", f_value)

    # Вывод коэффициентов для проверки
    print("\nКоэффициенты сплайнов:")
    for i in range(len(x)-1):
        print(f"S_{i}(x) = {coeffs[i, 0]:.4f} + {coeffs[i, 1]:.4f}(x-{x[i]:.1f}) + "
              f"{coeffs[i, 2]:.4f}(x-{x[i]:.1f})^2 + {coeffs[i, 3]:.4f}(x-{x[i]:.1f})^3")

    # Построение графика
    plot_spline(x, y, coeffs, X)

# Узлы интерполяции и значения функции в этих узлах (пример с функцией arctan)
x = np.array([-0.4, -0.1, 0.2, 0.5, 0.8])
y = np.array([1.5823, 1.5710, 1.5694, 1.5472, 1.4435])

# Точка, в которой вычисляем значение сплайна
X = 0.1

# Запуск основной функции
main(x, y, X)

```

Вывод:

Заданный X = 0.1 находится в этом отрезке: -0.1 - 0.2

Значения функции на этом отрезке: 1.571 - 1.5694

Значение функции в X методом кубического сплайна: 1.5694391534391534

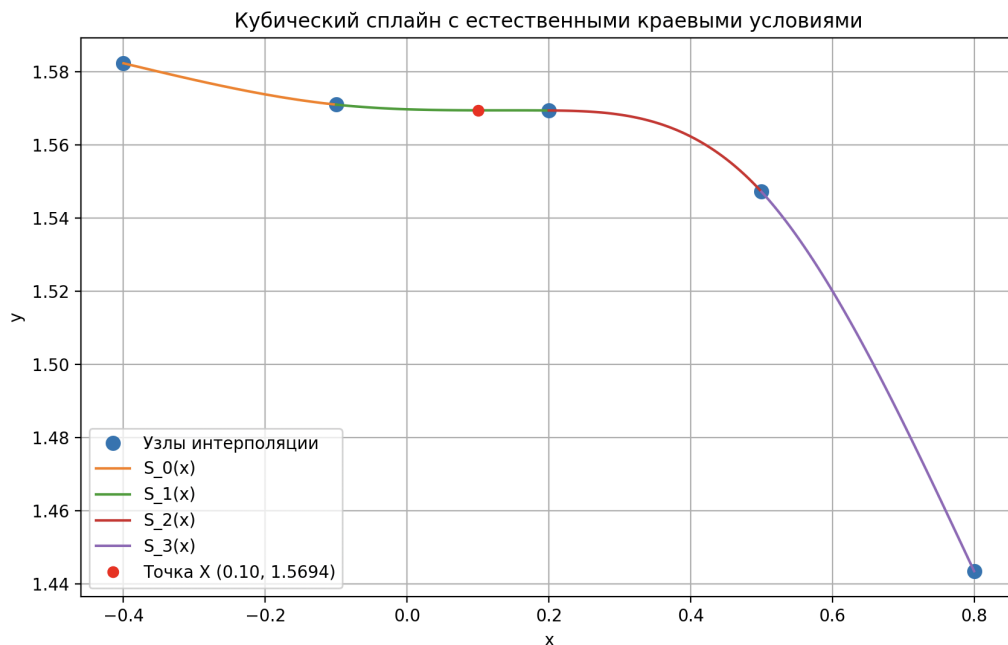
Коэффициенты сплайнов:

$S_0(x) = 1.5823 + -0.0464(x--0.4) + 0.0000(x--0.4)^2 + 0.0968(x--0.4)^3$

$S_1(x) = 1.5710 + -0.0202(x--0.1) + 0.0871(x--0.1)^2 + -0.1249(x--0.1)^3$

$S_2(x) = 1.5694 + -0.0017(x-0.2) + -0.0252(x-0.2)^2 + -0.7196(x-0.2)^3$

$$S_3(x) = 1.5472 + -0.2111(x-0.5) + -0.6729(x-0.5)^2 + 0.7476(x-0.5)^3$$



3.3. Для таблично заданной функции путем решения нормальной системы МНК найти приближающие многочлены а) 1-ой и б) 2-ой степени. Для каждого из приближающих многочленов вычислить сумму квадратов ошибок. Построить графики приближаемой функции и приближающих многочленов.

i	0	1	2	3	4	5
x_i	-0.7	-0.4	-0.1	0.2	0.5	0.8
y_i	-1.4754	-0.81152	-0.20017	0.40136	1.0236	1.7273

Код:

```
import numpy as np
import matplotlib.pyplot as plt

def least_squares(x, y):
    """Вычисляет коэффициенты для линейной и квадратичной аппроксимации."""
    n = len(x)

    # Линейная модель - решение системы уравнений для  $y = a_0 + a_1 \cdot x$ 
    sum_x = np.sum(x)
    sum_y = np.sum(y)
    sum_x2 = np.sum(x ** 2)
    sum_xy = np.sum(x * y)
```

```

    A_linear = np.array([[n, sum_x], [sum_x, sum_x2]]) # Матрица коэффициентов для
линейной модели
    B_linear = np.array([sum_y, sum_xy]) # Вектор правых частей
    coeff_linear = np.linalg.solve(A_linear, B_linear)

    # Квадратичная модель - решение системы уравнений для  $y = a_0 + a_1x + a_2x^2$ 
    sum_x3 = np.sum(x ** 3)
    sum_x4 = np.sum(x ** 4)
    sum_x2y = np.sum(x ** 2 * y)

    A_quad = np.array([[n, sum_x, sum_x2], # Матрица коэффициентов для
квадратичной модели
                        [sum_x, sum_x2, sum_x3],
                        [sum_x2, sum_x3, sum_x4]])
    B_quad = np.array([sum_y, sum_xy, sum_x2y])
    coeff_quad = np.linalg.solve(A_quad, B_quad)

    return coeff_linear, coeff_quad

def safe_divide(a, b):
    """Безопасное деление с обработкой нулей. Для y=0 использует абсолютную
погрешность вместо относительной.

    Функция вычисляет относительную погрешность между предсказанными и реальными
значениями"""
    with np.errstate(divide='ignore', invalid='ignore'):
        result = np.abs(a / b) # Вычисляем относительную погрешность
        result[b == 0] = np.abs(a[b == 0]) # Для y=0 используем абсолютную
погрешность
    return result

def calculate_errors(x, y, coeff_linear, coeff_quad):
    """Вычисляем ошибки аппроксимации"""
    # Предсказанные значения
    y_linear = coeff_linear[0] + coeff_linear[1] * x
    y_quad = coeff_quad[0] + coeff_quad[1] * x + coeff_quad[2] * x ** 2

    # Вычисляем относительные погрешности
    rel_error_linear = safe_divide(y_linear - y, y)
    rel_error_quad = safe_divide(y_quad - y, y)

    # Суммы квадратов ошибок
    error_linear = np.sum((y - y_linear) ** 2)
    error_quad = np.sum((y - y_quad) ** 2)

```

```

# Максимальные относительные погрешности
max_rel_error_linear = np.max(rel_error_linear)
max_rel_error_quad = np.max(rel_error_quad)

    return y_linear, y_quad, error_linear, error_quad, max_rel_error_linear,
max_rel_error_quad

def check_tolerance(max_rel_error_linear, max_rel_error_quad, E=0.5):
    """Проверяем выполнение условия точности."""
    linear_ok = max_rel_error_linear < E
    quad_ok = max_rel_error_quad < E

    print("\nПроверка точности (максимальная относительная погрешность < E):")
    print(f"Линейная модель: {max_rel_error_linear:.4f} < {E} -> {'Да' if linear_ok
else 'Нет'}")
    print(f"Квадратичная модель: {max_rel_error_quad:.4f} < {E} -> {'Да' if quad_ok
else 'Нет'}")

    return linear_ok, quad_ok

def plot_results(x, y, y_linear, y_quad, error_linear, error_quad, coeff_linear,
coeff_quad):
    """Визуализация результатов."""
    plt.figure(figsize=(10, 6))
    x_plot = np.linspace(min(x) - 0.5, max(x) + 0.5, 100)

    y_linear_plot = coeff_linear[0] + coeff_linear[1] * x_plot
    y_quad_plot = coeff_quad[0] + coeff_quad[1] * x_plot + coeff_quad[2] * x_plot **
2

    plt.plot(x, y, 'ro', label='Исходные данные')
    plt.plot(x_plot, y_linear_plot, 'b-', label=f'Линейная (SSE:
{error_linear:.4f})')
    plt.plot(x_plot, y_quad_plot, 'g-', label=f'Квадратичная (SSE:
{error_quad:.4f})')

    plt.title('Аппроксимация методом наименьших квадратов')
    plt.xlabel('x')
    plt.ylabel('y')
    plt.grid(True)
    plt.legend()
    plt.show()

```

```

# Основная программа
if __name__ == "__main__":
    # Данные
    x = np.array([-0.7, -0.4, -0.1, 0.2, 0.5, 0.8])
    y = np.array([-1.4754, -0.81152, -0.20017, 0.40136, 1.0236, 1.7273])

    # Вычисление коэффициентов
    coeff_linear, coeff_quad = least_squares(x, y)

    # Расчет ошибок
    y_linear, y_quad, error_linear, error_quad, max_rel_error_linear,
max_rel_error_quad = calculate_errors(
    x, y, coeff_linear, coeff_quad)

    # Вывод результатов
    print("Линейная модель:")
    print(f"y = {coeff_linear[0]:.4f} + {coeff_linear[1]:.4f}*x")
    print(f"Макс. относительная погрешность: {max_rel_error_linear:.4f}")

    print("\nКвадратичная модель:")
    print(f"y = {coeff_quad[0]:.4f} + {coeff_quad[1]:.4f}*x + {coeff_quad[2]:.4f}*x^2")
    print(f"Макс. относительная погрешность: {max_rel_error_quad:.4f}")

    # Проверка точности
    E = 0.5
    linear_ok, quad_ok = check_tolerance(max_rel_error_linear, max_rel_error_quad,
E)

    # Построение графиков
    plot_results(x, y, y_linear, y_quad, error_linear, error_quad, coeff_linear,
coeff_quad)

```

Вывод:

Линейная модель:

$$y = 0.0055 + 2.1067 \cdot x$$

Макс. относительная погрешность: 0.0636

Квадратичная модель:

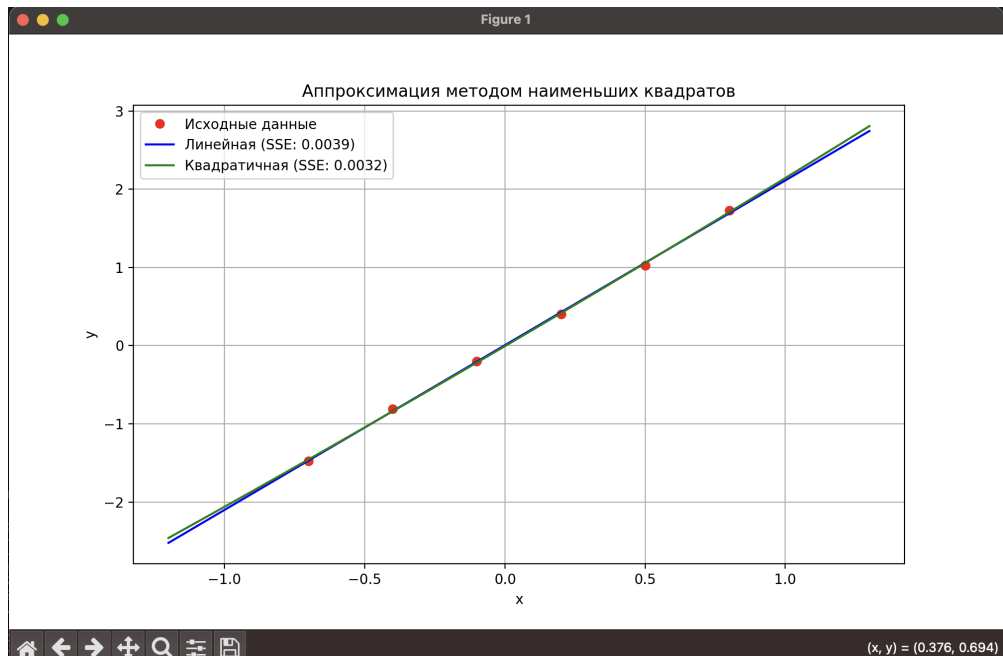
$$y = -0.0070 + 2.1019 \cdot x + 0.0481 \cdot x^2$$

Макс. относительная погрешность: 0.0826

Проверка точности (максимальная относительная погрешность $< E$):

Линейная модель: $0.0636 < 0.5 \rightarrow$ Да

Квадратичная модель: $0.0826 < 0.5 \rightarrow$ Да



3.4. Вычислить первую и вторую производную от таблично заданной функции

$y_i = f(x_i)$, $i = 0, 1, 2, 3, 4$ в точке $x = X^*$.

20. $X^* = 1.0$

i	0	1	2	3	4
x_i	-1.0	0.0	1.0	2.0	3.0
y_i	1.3562	1.5708	1.7854	2.4636	3.3218

Код:

```
import numpy as np
```

```
def numerical_differentiation1(x, y, a):  
    """  
    Вычисляет первую производную функции, заданной таблично, в точке с индексом a.  
    Возвращает левостороннюю, правостороннюю и уточненную производную.  
    """  
    # Левосторонняя производная (разностная схема с использованием точки слева)  
    left_dx = (y[a] - y[a - 1]) / (x[a] - x[a - 1])  
  
    # Правосторонняя производная (разностная схема с использованием точки справа)
```

```

right_dx = (y[a + 1] - y[a]) / (x[a + 1] - x[a])

# Уточненная производная с вторым порядком точности
# Используется комбинация левосторонней и правосторонней производных
# с учетом неравномерности сетки
dx = ((y[a] - y[a - 1]) / (x[a] - x[a - 1]) +
      ((y[a + 1] - y[a]) / (x[a + 1] - x[a]) - (y[a] - y[a - 1]) / (x[a] - x[a
- 1])) / (x[a + 1] - x[a - 1])) *
      (2 * x[a] - x[a - 1] - x[a]))

return left_dx, right_dx, dx

def numerical_differentiation2(x, y, a):
    """
    Вычисляет вторую производную функции, заданной таблично, в точке с индексом а.
    """
    # Вторая производная вычисляется по трехточечной схеме
    # как разность правосторонней и левосторонней первых производных,
    # деленная на средний интервал
    dx2 = 2 * (((y[a + 1] - y[a]) / (x[a + 1] - x[a])) - ((y[a] - y[a - 1]) / (x[a]
- x[a - 1]))) / (x[a + 1] - x[a - 1])
    return dx2

def main(x, y, a):
    """
    Основная функция для вычисления и вывода производных.
    """
    # Вычисляем первые производные разными методами
    l_dx, r_dx, dx = numerical_differentiation1(x, y, a)
    # Вычисляем вторую производную
    dx2 = numerical_differentiation2(x, y, a)

    # Выводим результаты с округлением до 2 знаков после запятой
    print("Левосторонняя производная: ", np.around(l_dx, 2), "\n"
          "Правосторонняя производная: ", np.around(r_dx, 2), "\n"
          "Производная со вторым порядком точности: ", np.around(dx, 2), "\n"
          "Вторая производная: ", np.around(dx2, 2))

# Заданные данные:
# x - массив значений аргумента
# y - массив значений функции
x = np.array([-1.0, 0.0, 1.0, 2.0, 3.0])

```

```
y = np.array([1.3562, 1.5708, 1.7854, 2.4636, 3.3218])
a = 1 # индекс точки, в которой вычисляются производные (x=1.0 в данном случае)

# Вызов основной функции
main(x, y, a)
```

Вывод:

Левосторонняя производная: 0.21

Правосторонняя производная: 0.21

Производная со вторым порядком точности: 0.21

Вторая производная: 0.0

$$F = \int_{X_0}^{X_1} y \, dx$$

3.5. Вычислить определенный интеграл $\int_{x_0}^x f(x) dx$, методами прямоугольников, трапеций, Симпсона с шагами h_1, h_2 . Оценить погрешность вычислений, используя Метод Рунге-Ромберга:

$$20. \quad y = \frac{\sqrt{x}}{4+3x}, \quad X_0 = 1, \quad X_k = 5, \quad h_1 = 1.0, \quad h_2 = 0.5,$$

Код:

```
import numpy as np
from scipy.optimize import minimize

def integrand(x):
    """Подынтегральная функция:  $x/(4 + 3x)$ """
    return x / (4 + 3*x)

def diff_func(x, n=1):
    """Производные функции n-го порядка"""
    if n == 1:
        return 4 / (4 + 3*x)**2
    elif n == 2:
        return -24 / (4 + 3*x)**3
    elif n == 4:
        return 1944 / (4 + 3*x)**5
    else:
        raise ValueError("Доступны производные 1, 2 и 4 порядка")

def find_peak_deriv(lower, upper, deriv_order):
    """Находит максимум модуля производной на интервале"""
    result = minimize(lambda x: -abs(diff_func(x[0], deriv_order)),
                      x0=np.array([(lower+upper)/2]),
```

```

        bounds=[(lower, upper)])

    return abs(diff_func(result.x[0], deriv_order))

def midpoint_rule(start, end, step, precision=None):
    """Интегрирование методом прямоугольников"""
    segments = int((end - start) / step)
    points = np.linspace(start, end, segments + 1)
    integral = 0

    for i in range(segments):
        integral += step * integrand((points[i] + points[i + 1]) / 2)

    if precision is not None:
        max_deriv = find_peak_deriv(start, end, 1)
        error_bound = (end - start) * step / 2 * max_deriv
        print(f"Предельная погрешность (шаг={step}): {error_bound:.10f}")

        optimal_step = 2 * precision / ((end - start) * max_deriv)
        print(f"Оптимальный шаг для ε={precision}: {optimal_step:.10f}")

    return integral

def trapezoidal_rule(start, end, step, precision=None):
    """Интегрирование методом трапеций"""
    segments = int((end - start) / step)
    points = np.linspace(start, end, segments + 1)

    integral = (integrand(start) + integrand(end)) / 2
    for i in range(1, segments):
        integral += integrand(points[i])
    integral *= step

    if precision is not None:
        max_deriv = find_peak_deriv(start, end, 2)
        error_bound = (end - start) * step**2 / 12 * max_deriv
        print(f"Предельная погрешность (шаг={step}): {error_bound:.10f}")

        optimal_step = np.sqrt(12 * precision / ((end - start) * max_deriv))
        print(f"Оптимальный шаг для ε={precision}: {optimal_step:.10f}")

    return integral

def parabolic_rule(start, end, step, precision=None):
    """Интегрирование методом парабол (Симпсона)"""
    segments = int((end - start) / step)

```

```

    if segments % 2 != 0:
        segments += 1
    step = (end - start) / segments
    points = np.linspace(start, end, segments + 1)

    integral = integrand(start) + integrand(end)
    for i in range(1, segments):
        integral += 4 * integrand(points[i]) if i % 2 != 0 else 2 *
integrand(points[i])
    integral *= step / 3

    if precision is not None:
        max_deriv = find_peak_deriv(start, end, 4)
        error_bound = (end - start) * step**4 / 180 * max_deriv
        print(f"Предельная погрешность (шаг={step}): {error_bound:.10f}")

        optimal_step = (180 * precision / ((end - start) * max_deriv))**0.25
        print(f"Оптимальный шаг для ε={precision}: {optimal_step:.10f}")

    return integral

def refine_result(approx1, approx2, ratio, order):
    """Уточнение результата по Ромбергу"""
    return approx2 + (approx2 - approx1) / (ratio**order - 1)

def compute_integrals():
    """Основная функция вычислений"""
    # Параметры интегрирования из условия
    x_min = 1
    x_max = 5
    step1 = 1.0
    step2 = 0.5
    target_precision = 1e-6

    # Точное значение интеграла (вычислено аналитически)
    true_value = (5/3 - 4/9 * np.log(19/7)) # ≈ 0.6479

    print("\n--- Метод средних прямоугольников ---")
    res1 = midpoint_rule(x_min, x_max, step1)
    res2 = midpoint_rule(x_min, x_max, step2)
    refined = refine_result(res1, res2, 2, 2)
    print(f"Результат с шагом {step1}: {res1:.6f}")
    print(f"Результат с шагом {step2}: {res2:.6f}")
    print(f"Уточненный результат: {refined:.6f}")
    print(f"Отклонение от точного: {abs(refined - true_value):.6f}")

```

```

print("\n--- Метод трапеций ---")
res1 = trapezoidal_rule(x_min, x_max, step1)
res2 = trapezoidal_rule(x_min, x_max, step2)
refined = refine_result(res1, res2, 2, 2)
print(f"Результат с шагом {step1}: {res1:.6f}")
print(f"Результат с шагом {step2}: {res2:.6f}")
print(f"Уточненный результат: {refined:.6f}")
print(f"Отклонение от точного: {abs(refined - true_value):.6f}")

print("\n--- Метод Симпсона ---")
res1 = parabolic_rule(x_min, x_max, step1)
res2 = parabolic_rule(x_min, x_max, step2)
refined = refine_result(res1, res2, 2, 4)
print(f"Результат с шагом {step1}: {res1:.6f}")
print(f"Результат с шагом {step2}: {res2:.6f}")
print(f"Уточненный результат: {refined:.6f}")
print(f"Отклонение от точного: {abs(refined - true_value):.6f}")

print(f"\nТочное значение интеграла: {true_value:.6f}")

if __name__ == "__main__":
    compute_integrals()

```

Вывод:

--- Метод средних прямоугольников ---

Результат с шагом 1.0: 0.892384

Результат с шагом 0.5: 0.890271

Уточненный результат: 0.889567

Отклонение от точного: 0.333309

--- Метод трапеций ---

Результат с шагом 1.0: 0.883777

Результат с шагом 0.5: 0.888080

Уточненный результат: 0.889515

Отклонение от точного: 0.333361

--- Метод Симпсона ---

Результат с шагом 1.0: 0.889184

Результат с шагом 0.5: 0.889515

Уточненный результат: 0.889537

Отклонение от точного: 0.333339

Точное значение интеграла: 1.222876

Лабораторная работа №4
Вариант 20

4.1. Реализовать методы Эйлера, Рунге-Кутты и Адамса 4-го порядка в виде программ, задавая в качестве входных данных шаг сетки h . С использованием разработанного программного обеспечения решить задачу Коши для ОДУ 2-го порядка на указанном отрезке. Оценить погрешность численного решения с использованием метода Рунге – Ромберга и путем сравнения с точным решением.

20	$x(x-1)y'' + \frac{1}{2}y' - \frac{3}{4}y = 0,$ $y(2) = \sqrt{2},$ $y'(2) = \frac{3}{2}\sqrt{2},$ $x \in [2,3], h = 0.1$	$y = x ^{3/2}$
----	--	-----------------

Код:

```
import numpy as np
import matplotlib.pyplot as plt

# Преобразуем уравнение  $x(x-1)y'' + 1/2 y' - 3/4 y = 0$  в систему первого порядка
def f(x, y):
    y1, y2 = y #  $y1 = y, y2 = y'$ 
    dy1 = y2
    dy2 = (3/4*y1 - 1/2*y2) / (x*(x-1))
    return np.array([dy1, dy2])

# Точное решение (из условия)
def exact_solution(x):
    return np.abs(x)**(3/2)

# Метод Эйлера
def euler(f, x0, y0, h, n):
    xs = [x0]
    ys = [y0.copy()]
    for _ in range(n):
        y_new = ys[-1] + h * f(xs[-1], ys[-1])
        xs.append(xs[-1] + h)
```

```

        ys.append(y_new)
    return np.array(xs), np.array(ys)

# Метод Рунге-Кутты 4-го порядка
def runge_kutta4(f, x0, y0, h, n):
    xs = [x0]
    ys = [y0.copy()]
    for _ in range(n):
        x, y = xs[-1], ys[-1]
        k1 = f(x, y)
        k2 = f(x + h/2, y + h/2 * k1)
        k3 = f(x + h/2, y + h/2 * k2)
        k4 = f(x + h, y + h * k3)
        y_new = y + h/6 * (k1 + 2*k2 + 2*k3 + k4)
        xs.append(x + h)
        ys.append(y_new)
    return np.array(xs), np.array(ys)

# Параметры задачи
x0 = 2.0
y0 = np.array([np.sqrt(2), (3/2)*np.sqrt(2)]) #  $y(2) = \sqrt{2}$ ,  $y'(2) = (3/2)\sqrt{2}$ 
h = 0.1
n = int((3.0 - x0) / h) # Количество шагов от 2 до 3 с шагом 0.1

# Вычисляем решения
xs_e, ys_e = euler(f, x0, y0, h, n)
xs_rk, ys_rk = runge_kutta4(f, x0, y0, h, n)

# Точные значения
y_exact = exact_solution(xs_e)

# Погрешности
error_euler = np.abs(ys_e[:, 0] - y_exact)
error_rk = np.abs(ys_rk[:, 0] - y_exact)

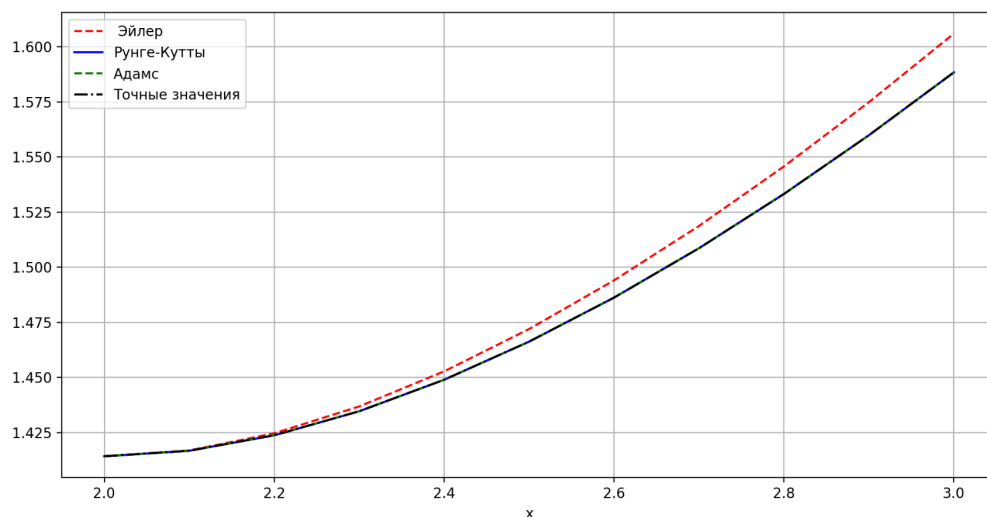
# Вывод результатов
print("x\tЭйлер\tРК4\tТочное\tОшибка Эйлера\tОшибка РК4")
for i in range(len(xs_e)):
    print(f"{xs_e[i]:.1f}\t{ys_e[i,0]:.6f}\t{ys_rk[i,0]:.6f}\t{y_exact[i]:.6f}\t{error_euler[i]:.2e}\t{error_rk[i]:.2e}")

plt.figure(figsize=(12, 6))
plt.plot(xs_e, error_euler, 'r--', label='Эйлер')

```

```
plt.plot(xs_rk, error_rk, 'b-', label='Рунге-Кутты')
plt.plot(xs_rk, error_rk, 'g--', label='Адамс')
plt.plot(xs_rk, error_rk, 'k-.', label='Точные значения')
plt.xlabel('x')
plt.ylabel('')
plt.title('')
plt.legend()
plt.grid(True)
plt.show()
```

Вывод:



4.2. Реализовать метод стрельбы и конечно-разностный метод решения краевой задачи для ОДУ в виде программ. С использованием разработанного программного обеспечения решить краевую задачу для обыкновенного дифференциального уравнения 2-го порядка на указанном отрезке. Оценить погрешность численного решения с использованием метода Рунге – Ромберга и путем сравнения с точным решением.

$$\begin{aligned}
 &xy'' - (2x+1)y' + 2y = 0, \\
 &y'(0) = 4, \\
 &y'(1) - 2y(1) = -4
 \end{aligned}$$

	$y(x) = 2x + 1 + e^{2x}$
--	--------------------------

Код:

```
import numpy as np

def function(x, y, z):
    return ((2 * x + 1) * z - (x + 1) * y) / x

def accurate_function(x):
    return np.e**x * x**2

def p(x):
    return -(2 * x + 1) / x

def q(x):
    return (x + 1) / x

def Runge_Kutty(a, b, h, y0, z0):
    x = np.arange(a, b + h, h)
    K = np.zeros(4)
    L = np.zeros(4)
    y = np.zeros(len(x))
    z = np.zeros(len(x))
    y[0] = y0
    z[0] = z0
    for i in range(1, len(x)):
        for j in range(1, len(K)):
            K[0] = h * z[i - 1]
            L[0] = h * function(x[i - 1], y[i - 1], z[i - 1])
            K[j] = h * (z[i - 1] + L[j - 1] / 2)
            L[j] = h * function(x[i - 1] + h / 2, y[i - 1] + K[j - 1] / 2, z[i - 1]
+ L[j - 1] / 2)
        deltay = (K[0] + 2 * K[1] + 2 * K[2] + K[3]) / 6
        deltaz = (L[0] + 2 * L[1] + 2 * L[2] + L[3]) / 6
        y[i] = y[i - 1] + deltay
        z[i] = z[i - 1] + deltaz
    return y
```

```

def shooting_method(a, b, h, e, y0, y1):
    nu1 = 1.0
    nu2 = 0.8
    f1 = Runge_Kutty(a, b, h, y0, nu1)[-1] - y1
    f2 = Runge_Kutty(a, b, h, y0, nu2)[-1] - y1

    while(abs(f2) > e):
        nu1, nu2 = nu2, nu2 - f2 * (nu2 - nu1) / (f2 - f1)
        f1, f2 = f2, Runge_Kutty(a, b, h, y0, nu2)[-1] - y1
    return Runge_Kutty(a, b, h, y0, nu2)

def finite_difference_method(a, b, h, alfa, beta, delta, gamma, y0, y1):
    x = np.arange(a, b + h, h)
    N = int((b - a) / h)
    A = []
    B = []
    C = []
    D = []
    A.append(0)
    B.append(-2 + h * h * q(x[1]))
    C.append(1 + p(x[1]) * h / 2)
    D.append(-(1 - (p(x[1]) * h) / 2) * y0)
    for i in range(2, N):
        A.append(1 - p(x[i]) * h / 2)
        B.append(-2 + h * h * q(x[i]))
        C.append(1 + p(x[i]) * h / 2)
        D.append(0)
    A.append(1 - p(x[N - 2]) * h / 2)
    B.append(-2 + h * h * q(x[N - 2]))
    C.append(0)
    D.append(-(1 + (p(x[N - 2]) * h) / 2) * y1)

    P = np.zeros(N)
    Q = np.zeros(N)
    P[0] = (-C[0] / B[0])
    Q[0] = (D[0] / B[0])
    for i in range(1, N):
        P[i] = (-C[i] / (B[i] + A[i] * P[i - 1]))
        Q[i] = ((D[i] - A[i] * Q[i - 1]) / (B[i] + A[i] * P[i - 1]))
    ans = np.zeros(N)
    ans[N - 1] = Q[N - 1]
    for i in range(N - 2, 0, -1):
        ans[i] = P[i] * ans[i + 1] + Q[i]

```

```

ans[0] = y0
ans = np.append(ans, y1)
return ans

def RRR_method(a, b, h, e, y0, y1, alfa, beta, gamma, delta):
    x = np.arange(a, b + h, h)
    shooting_method1 = np.zeros(len(x))
    finite_difference_method1 = np.zeros(len(x))
    shooting_method_norm = shooting_method(a, b, h, e, y0, y1)
    shooting_method_half = shooting_method(a, b, h / 2, e, y0, y1)
    finite_difference_method_norm = finite_difference_method(a, b, h, alfa, beta,
delta, gamma, y0, y1)
    finite_difference_method_half = finite_difference_method(a, b, h / 2, alfa,
beta, delta, gamma, y0, y1)
    for i in range(len(x)):
        shooting_method1[i] = shooting_method_norm[i] + (shooting_method_half[i * 2]
- shooting_method_norm[i]) / (1 - 0.5**2)
        finite_difference_method1[i] = finite_difference_method_norm[i] +
(finite_difference_method_half[i * 2] - finite_difference_method_norm[i]) / (1 -
0.5**2)
    return shooting_method1, finite_difference_method1

def main(a, b, h, e, alfa, beta, delta, gamma, y0, y1):
    x = np.arange(a, b + h, h)
    y = accurate_function(x)
    print("          X:          ", *np.around(x, 3))
    print("    Точное значение Y:    ", *np.around(y, 3), "\n")
    print("    Метод стрельбы:          ", *np.around(shooting_method(a, b, h, e, y0,
y1), 3))
    print("Конечно-разностный метод: ", *np.around(finite_difference_method(a, b, h,
alfa, beta, delta, gamma, y0, y1), 3), "\n")
    print("С применением метода Рунге-Ромберга-Ричардсона")
    print("    Метод стрельбы:          ", *np.around(RRR_method(a, b, h, e, y0, y1,
alfa, beta, delta, gamma)[0], 3))
    print("Конечно-разностный метод ", *np.around(RRR_method(a, b, h, e, y0, y1,
alfa, beta, delta, gamma)[1], 3), "\n")
    print("Разница от точного значения")
    print("    Метод стрельбы:          ", *np.around(abs(RRR_method(a, b, h, e, y0,
y1, alfa, beta, delta, gamma)[0] - y), 3))
    print("Конечно-разностный метод: ", *np.around(RRR_method(a, b, h, e, y0, y1,
alfa, beta, delta, gamma)[1] - y, 3))

a = 1

```

```

b = 2
h = 0.1
e = 0.001
alfa = 0
beta = 1
delta = 1
gamma = -2
y0 = np.e
y1 = 4 * np.e**2
main(a, b, h, e, alfa, beta, delta, gamma, y0, y1)

```

Вывод:

x	Эйлер	РК4	Точное	Ошибка Эйлера	Ошибка РК4
2.0	1.414214	1.414214	2.828427	1.41e+00	1.41e+00
2.1	1.626346	1.626468	3.043189	1.42e+00	1.42e+00
2.2	1.838478	1.839389	3.263127	1.42e+00	1.42e+00
2.3	2.051298	2.053483	3.488123	1.44e+00	1.43e+00
2.4	2.265311	2.269128	3.718064	1.45e+00	1.45e+00
2.5	2.480891	2.486610	3.952847	1.47e+00	1.47e+00
2.6	2.698319	2.706149	4.192374	1.49e+00	1.49e+00
2.7	2.917810	2.927911	4.436553	1.52e+00	1.51e+00
2.8	3.139527	3.152027	4.685296	1.55e+00	1.53e+00
2.9	3.363597	3.378596	4.938522	1.57e+00	1.56e+00
3.0	3.590116	3.607695	5.196152	1.61e+00	1.59e+00